Eng 3.6. Network Security Protocols

Task 1 - Introduction

A network protocol specifies how two devices, or more precisely processes, communicate with each other. A network protocol is a pre-defined set of rules and processes to determine how data is transmitted between devices, such as end-user devices, networking devices, and servers. The fundamental objective of all protocols is to allow machines to connect and communicate seamlessly.

Task 2 - Application Layer

Each protocol represents specific layers of the OSI or TCP/IP model and operates as per the functionality of that layer. TCP and UDP based protocols operate on specific network ports.

HTTPS Protocol: HTTPS (Hypertext Transfer Protocol Secure) is the secure version of HTTP, used to safely transmit data between a web browser and a web server. Unlike HTTP, which sends data in plain text, HTTPS encrypts communication using SSL/TLS.

HTTPS is essential when handling sensitive information such as passwords, personal details, online banking transactions, and credit card payments. While HTTP traffic can be intercepted and read by attackers, HTTPS protects data from eavesdropping and tampering.

HTTPS works the same way as HTTP but adds an encryption layer. It uses asymmetric encryption (public and private keys) to establish a secure connection and exchange a symmetric encryption key for faster data encryption during the session.

HTTP Port: 80
HTTPS Port: 443
Encrypts data in transit

Protects confidentiality and integrity
Prevents attackers from reading or modifying transmitted information

Without HTTPS, secure online services such as banking, e-commerce, and online payments would not be possible.

FTPS Protocol: FTPS (File Transfer Protocol Secure) is a secure version of FTP that adds SSL/TLS encryption to protect authentication credentials and file transfers. While traditional FTP sends usernames, passwords, and data in plain text, FTPS encrypts communications to prevent interception and unauthorized access.
Control Connection (TCP 21): Used for authentication and sending commands.
Data Connection: Used for transferring files and folders.

FTP Modes
Active Mode: Client connects to the server on port 21, and the server initiates the data connection back to the client on port 20. This can be blocked by client-side firewalls.
Passive Mode: The client establishes both control and data connections, making it firewall-friendly and more commonly used today.

Data Transfer Types
ASCII (Type A): Text files.
Binary/Image (Type I): Files transferred byte-for-byte.
EBCDIC (Type E): Text transfer using the EBCDIC character set.
Local Type L: Used by systems with non-standard byte sizes.

FTPS Security Modes
Implicit FTPS (Port 990): Encryption is required immediately upon connection.
Explicit FTPS (Port 21): The client requests SSL/TLS encryption after connecting. It can encrypt the control channel, data channel, or both.

Encrypts usernames, passwords, and transferred files.
Protects against packet sniffing and data theft.
Maintains the functionality of FTP while adding security.

Ports:FTP: 21
Explicit FTPS: 21
Implicit FTPS: 990

SMTPS Protocol: SMTPS (Simple Mail Transfer Protocol Secure) is a secure version of SMTP that uses **SSL/TLS encryption** to protect email communications. While SMTP is responsible for sending emails, SMTPS adds confidentiality, integrity, and authentication to prevent unauthorized access and interception.

How SMTP Works: SMTP is an **"Email Push Protocol"** used to send emails from a client to a mail server. It operates in two models:

- **End-to-End SMTP:** Sends emails directly between organizations.
- **Store-and-Forward SMTP:** The mail server temporarily stores emails before delivering them to recipients.

Key Components:

- **User Agent (UA):** Creates and sends email messages.
- **Mail Transfer Agent (MTA):** Transfers emails across the internet to the recipient's mail server.

SMTPS Security: SMTPS wraps SMTP traffic inside **TLS/SSL encryption**. The **STARTTLS** command upgrades a standard SMTP connection to a secure encrypted session.

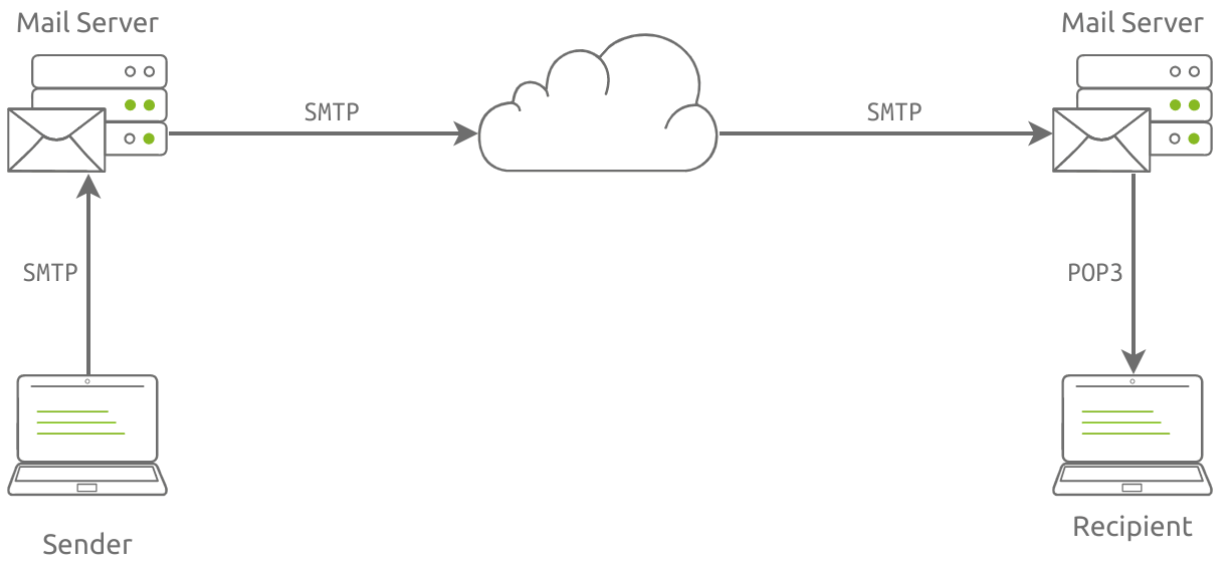
Common Ports

- **SMTP:** Port 25
- **SMTPS / Submission with TLS:** Ports **465** and **587**

Benefits of SMTPS

- Encrypts email content during transmission.
- Protects against packet sniffing and data theft.
- Ensures message confidentiality and integrity.
- Reduces risks of spam, phishing, and unauthorized email access.

Without SMTPS, emails sent via standard SMTP can be intercepted and read in plain text while traveling across networks.



POP3S Protocol: POP3S (Post Office Protocol Secure) is the secure version of POP3, used to **retrieve emails securely** from a mail server to an email client. It enhances POP3 by adding **TLS/SSL encryption**, protecting credentials and email content during transmission. How POP3 Works: POP3 is an email retrieval protocol that downloads messages from a mail server to a user's device.

1. The email client connects to the mail server.
2. Emails are downloaded to the client device.
3. Messages are stored locally.
4. By default, emails are deleted from the server after download (though this behavior can be changed).

Limitations of POP3

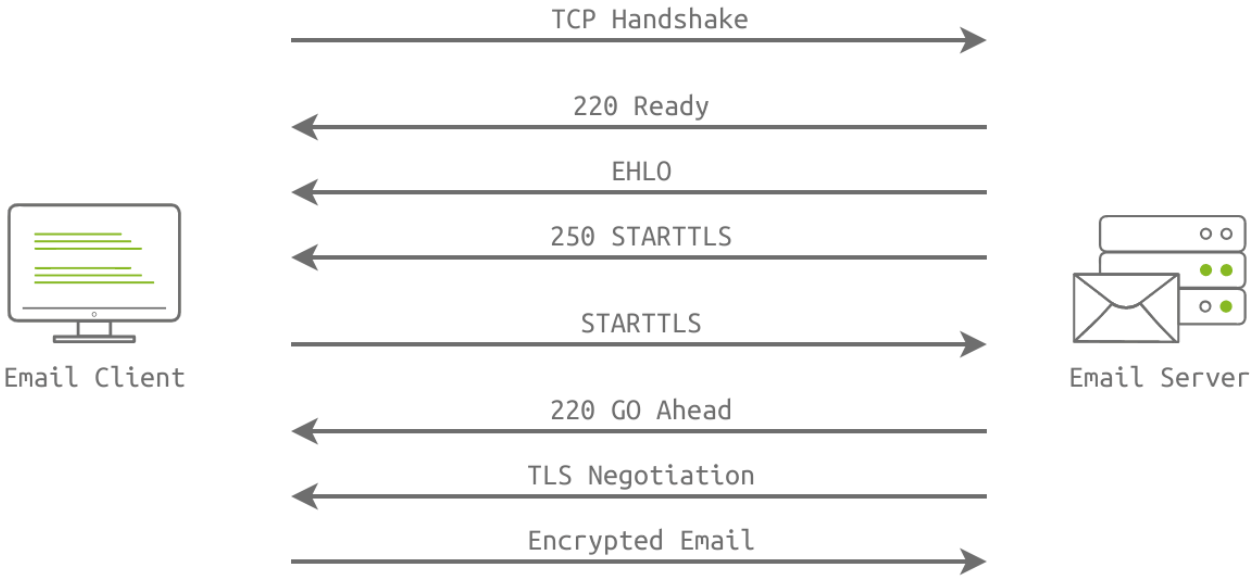
- **No synchronization:** Emails are typically stored on a single device, making access from multiple devices difficult.
- **Unencrypted communication:** Usernames, passwords, and email content can be transmitted in plain text, making them vulnerable to interception.

POP3S Security: POP3S secures email retrieval by wrapping POP3 traffic in **TLS/SSL encryption**. The client and server use the **STARTTLS** command to upgrade the connection to a secure encrypted session. Benefits of POP3S

- Encrypts usernames, passwords, and email content.

- Protects against packet sniffing and credential theft.
- Ensures confidentiality and integrity of email communications.
- Provides a more secure alternative to standard POP3.

In short, **POP3S = POP3 + TLS/SSL encryption**, enabling secure email retrieval from mail servers.



The POP3S Protocol uses port 995, while POP3 uses port 110.

Answer the questions below

> What is the default port for HTTPS?

443

> In a passive FTP connection, what does the client send the first command over the command channel?

PASV

> Use the SSL Shopper (opens in new tab) website to check the SSL certificate of TryHackMe.

> Open the SuperTool (opens in new tab) website, select the Test Email Server option, and check the SMTP Security for smtp.gmail.com.

Task 3 - Application Layer - More Secure Protocols

DNSSEC: DNS stands for Domain Name System. The DNS protocol is responsible mainly for resolving domain names. Instead of remembering the IP address, you need to focus on the domain name. For instance, at the time of this writing, `example.com` resolves to `93.184.216.34`; it is clear which one is easier for the human mind to remember. DNS works by sending a DNS query. For instance, when browsing the web, your web browser might send a query for DNS record type `A` or `AAAA`, i.e., IPv4 or IPv6 addresses. In the following console output, we can see the host with IP address `192.168.0.102` sending two DNS queries to the DNS server `1.0.0.1` regarding the domain name `example.com`. We can see the responses for the `A` and `AAAA` queries.

DNSSEC makes it possible to ensure that the DNS response we receive is from the domain owner. To achieve this, DNSSEC requires two main things:

1. The DNS zone owner should sign all DNS records using their private key.
2. The DNS zone publishes its public key so users can check the validity of the DNS records signatures.

In other words, the data to our DNS query is signed to ensure its integrity and authenticity; moreover, we can efficiently check the signature. With signed records, DNSSEC provides the following:

- Authenticity: You can confirm that a certain DNS owner has authored and sent the record. Authenticity is possible because the received record is signed by the DNS owner's private key.
- Integrity: You can ensure that no changes have been made to the record on its way. Any changes to the record will render its signature invalid.

OpenPGP (Pretty Good Privacy): OpenPGP is an open standard for **encrypting and digitally signing emails and files**. While protocols such as **SMTPS, POP3S, IMAPS, and HTTPS** encrypt data during transmission, mail servers can still access the contents of emails. OpenPGP provides **end-to-end encryption**, ensuring that only the intended recipient can read the message. Why OpenPGP?

- SSL/TLS protects data **in transit**.
- Mail servers can still decrypt and read emails.

- OpenPGP encrypts the email itself, preventing even mail servers from viewing its contents.

GnuPG (GPG): GnuPG is a free and open-source implementation of the OpenPGP standard. It allows users to:

- Encrypt emails and files.
- Digitally sign messages.
- Verify the authenticity of received communications.

How OpenPGP Works: Each user generates a **key pair**:

- **Public Key:** Shared with others and used to encrypt messages.
- **Private Key:** Kept secret and used to decrypt messages and create digital signatures.

Email Encryption Process

1. The sender encrypts the message using the recipient's **public key**.
2. The sender signs the message using their **private key**.
3. The recipient verifies the signature using the sender's **public key**.
4. The recipient decrypts the message using their **private key**.

Common GPG Commands: Generate a key pair:

```
gpg--gen-key
```

Encrypt and sign a message:

```
gpg--encrypt--sign--armor-r recipient@example.com message.txt
```

- `-encrypt -r recipient@example.com` → Encrypts the file using the recipient's public key (**confidentiality**).
- `-sign` → Signs the message with the sender's private key (**authenticity and integrity**).

- `-armor` → Produces ASCII output instead of binary for easier email transmission.

Benefits:

✅ End-to-end encryption

✅ Protects email confidentiality

✅ Verifies sender identity

✅ Detects message tampering

✅ Mail servers cannot read encrypted content

❌ Email **headers** (sender, recipient, subject, timestamps) are generally not encrypted and remain visible.

In short: OpenPGP adds true end-to-end security to email by combining **public-key encryption** and **digital signatures**, ensuring that only the intended recipient can read the message and verify its authenticity.

SSH: With the beginning of networked systems, there was a need to connect to a system over a network. For instance, a user needs to execute processes and administer the system. The aim was to do this remotely instead of being physically present at the computer.

Two of the earliest protocols were Telnet and remote login, with the clients `telnet` and `rlogin`, respectively. Both protocols made it possible to log in to a system over a network; however, neither focused on the security aspects. The confidentiality of the exchanged traffic, especially the login credentials, was not protected. Moreover, the integrity of the traffic and commands sent was not ensured. In other words, it was easy for an attacker monitoring the network to read the login credentials and modify the commands sent over the network.

The screenshot below is taken from Wireshark after using “Follow TCP Stream” against a Telnet session. We can see the user typing his username `michael` and pasting his password `RJ9wn^t3T%gC`. Although the password is secure by modern standards, it is sent in cleartext for any properly located network packet-capturing software to read. Note that in the image below, the text in red is sent by the Telnet client, while the text in blue is sent by the Telnet server.

The Secure Shell Protocol (SSH) provided the security requirements lacking in Telnet and remote login. With SSH, it is no longer feasible for the attacker to read the login credentials or modify the traffic. If we attempt to “Follow TCP Stream” on Wireshark after capturing the packets, we won’t get any information regarding the username, password, or issued commands. The screenshot below shows that the only visible information is the version numbers and the supported protocols. (Please note that the red characters are sent by the client, while the blue characters are sent by the server.)

Answer the questions below

> What does PGP stand for?

Pretty Good Privacy

> What does GPG stand for?

Gnu Privacy Guard

> What command would you use to generate a key pair using gpg?

gpg --gen-key

> Consider the following three clients:

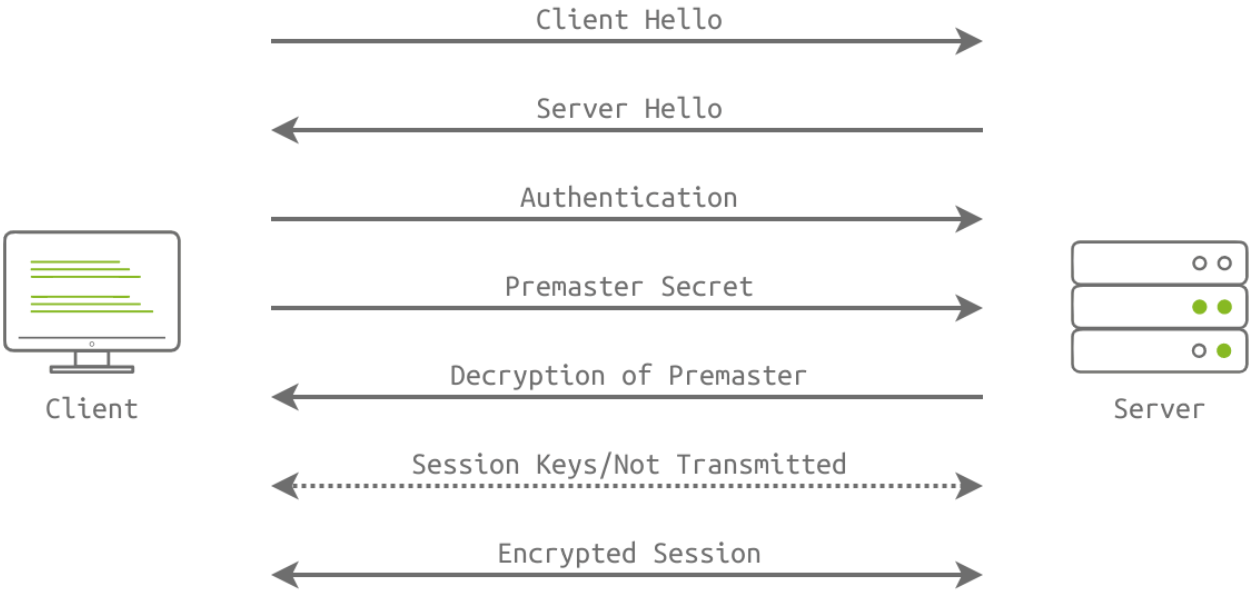
rlogin
telnet
ssh *
Provide the number of the client that encrypts the traffic.

Task 4 - Presentation and Session Layers

SSL/TLS Protocol: Secure Socket Layer (SSL) and Transport Layer Security (TLS) are protocols used to encrypt data exchanged between a client, such as a web browser, and a server. Consider SSL/TLS as a wrapper that encrypts various communication protocols, such as HTTP and FTP, to create HTTPS and FTPS. SSL is not commonly used nowadays as TLS has been gradually replacing it.

SSL/TLS Workflow: SSL/TLS handshake is performed to encrypt the communication between client and server through the following steps:

1. Client Hello Message: The client sends a hello message to the server; it includes the client TLS version and the cypher suite that the client supports, in addition to random bytes.
2. Server Hello Message: The server responds with a hello message, highlighting its certificate, chosen cypher suite and random bytes.
3. Authentication: The client authenticates the server's certificate through the certificate authority that issued it. For example, when we visit Google(opens in new tab), Google shares its certificate. The received certificate is verified by our browser, which is pre-installed with the certificates of various certificate authorities.
4. Premaster Secret: The client encrypts random bytes with the server's public key. (The client retrieves the public key from the server's certificate.)
5. Decryption of Premaster: The server decrypts the premaster with its private key.
6. Session Keys Generated: The client and the server generate session keys based on client random bytes, random server bytes and premaster secret. Both will arrive at the same results; **this session key is not transmitted**, and encryption and decryption are based on this key.
7. Ready Messages: The client and server send a "finished" message using the session key to indicate that the session is ready for transmission. The client and server are now ready to exchange messages over SSL/TLS encrypted connection.



TLS is a wrapper that encrypts communication of communication protocols. It has port numbers for various protocols, such as 443 for HTTPS and 990 for FTPS.

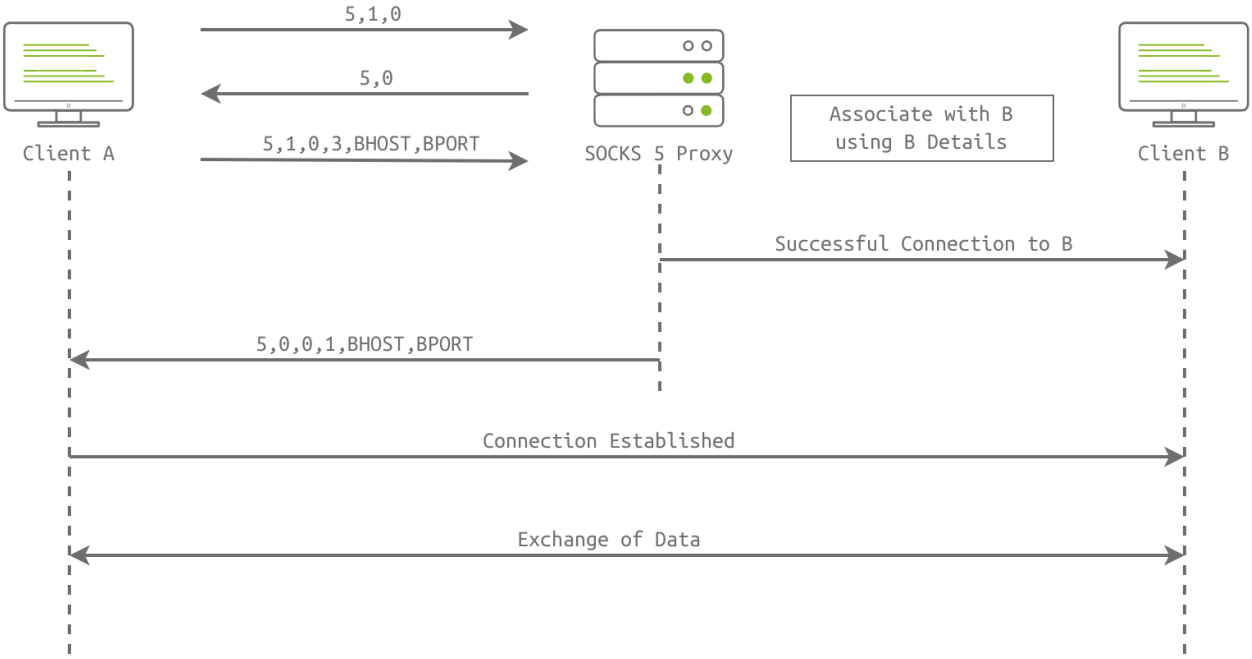
SOCKS5 Protocol: Socket Secure (SOCKS) is a proxy protocol for data exchange through a delegate server (SOCKS5 proxy). It is used to secure application layer protocols. For example, the Squid server implements the SOCKS5 protocol to transfer data via the HTTP protocol.

SOCKS5 Workflow: Consider a scenario when user A wants to connect with client B over the Internet, but a firewall is between them. The following handshake steps are involved:

- **Client Initiation**
  - Client A connects with the SOCKS5 proxy and sends the first byte (0x05) to the proxy where “5” is the SOCKS version.
  - Client A sends a second byte (0x01). One means authentication is supported.
  - Client A sends the third byte (0x00, 0x01, 0x02, or 0x03); these bytes denote the supported authentication methods and can be of variable length.
- **SOCKS5 Proxy Reply**

- The proxy sends back a second byte, which is the chosen authentication method by the proxy server.
- After the initiation packet, client A sends the request packet, which includes BHOST & BPORT numbers.
- The successful session is established between client A and the proxy. The same steps are involved in the association of client B with the proxy.

- **Data Transfer**
  - After successfully associating both clients with a proxy server, both clients can exchange data and share information that will be routed through the proxy server.



Benefits of SOCKS5

- In direct communication via the proxy server, hide the internal details from routing over the Internet.
- A proxy acts as a relay server, bypassing Internet censorship based on the client's IP address.

Answer the questions below

> Does the hello message during the SSL handshake include the TLS version?

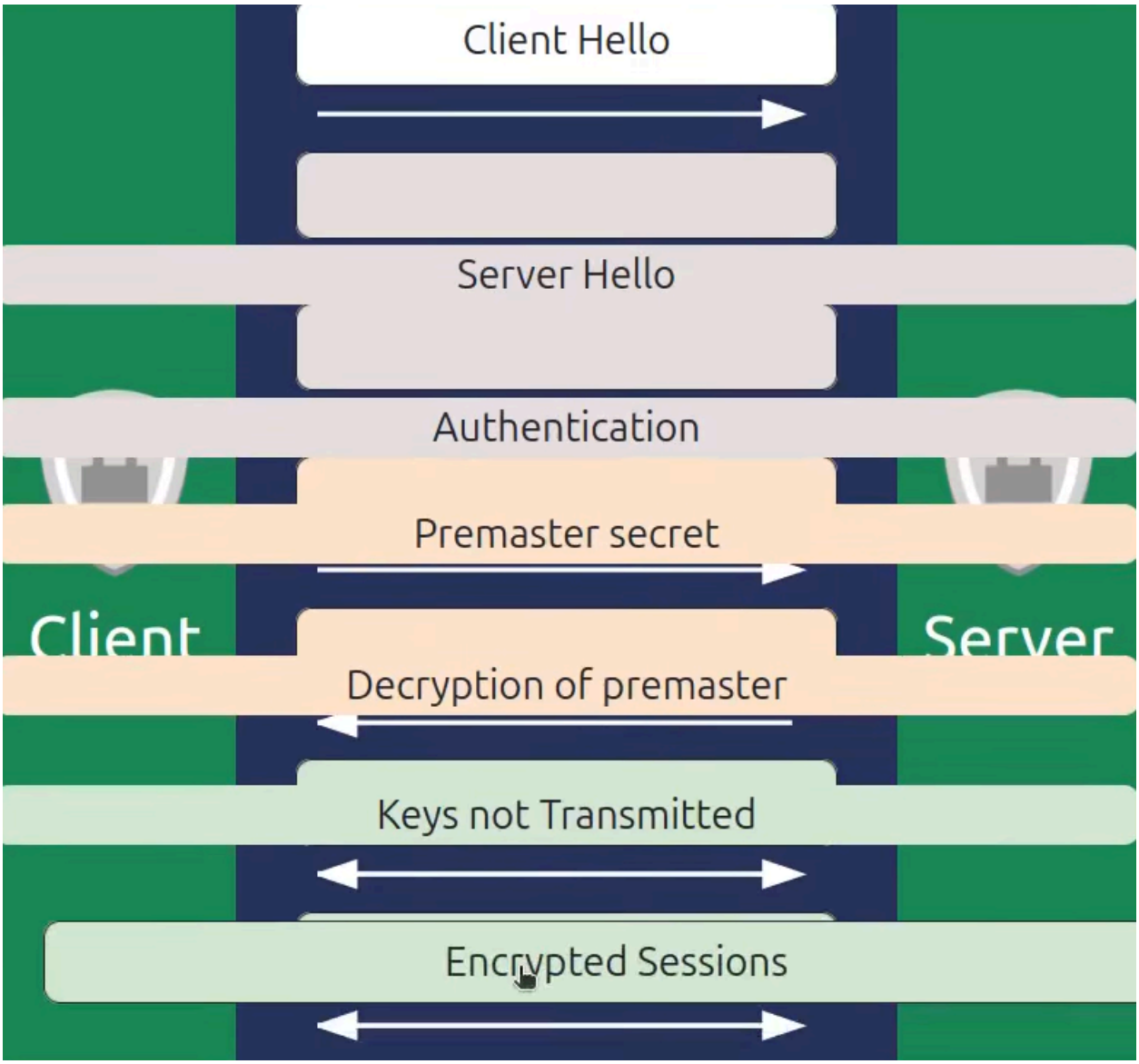
yea

> During the client initiation process of SOCKS5, what is the SOCKS version if the client sends the first 5 bytes (0x05)?

5

> Click the View Site button at the top of the task to launch the static site in split view. What is the flag after completing the exercise?

THM{GOT_THE_SSLKEY}



Task 5 - Network Layer

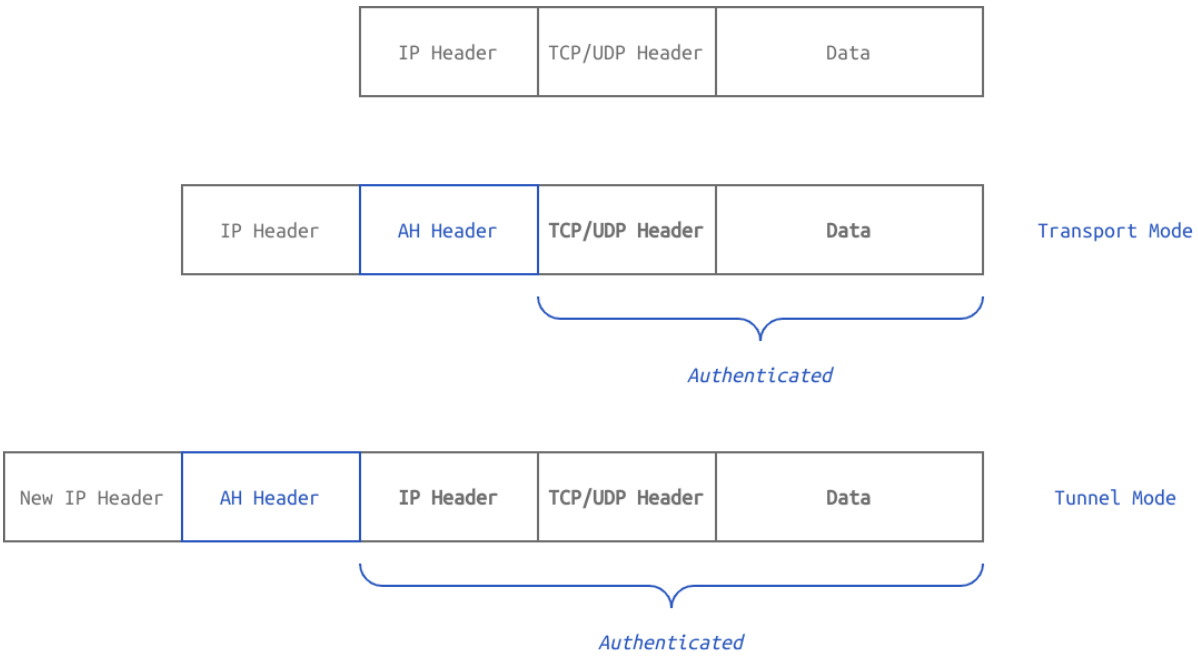
IPsec: IPsec stands for Internet Protocol Security. In this room, we use IPsec to refer to IPsec-v3. IPsec provides security by adding authentication and protecting the integrity and confidentiality of the network traffic. IPsec uses the following protocols:

1. Authentication Header (AH): Provides authentication and integrity.
2. Encapsulating Security Payload (ESP): Provides authentication, integrity, and confidentiality.

3. Security Association (SA): Is responsible for negotiating the encryption keys and algorithms. One example is Internet Key Exchange (IKE). Discussing SA in more detail is outside the scope of this room.

Authentication Header (AH): Authentication Header (AH): The AH protocol is responsible for the authentication and the integrity of the traffic; however, it cannot protect the confidentiality of the data. The AH protocol works in two modes, as shown in the figure below:

1. Transport Mode: Provides authentication for the TCP/UDP header and data.
2. Tunnel Mode: Provides authentication for the IP header, TCP/UDP header, and data.

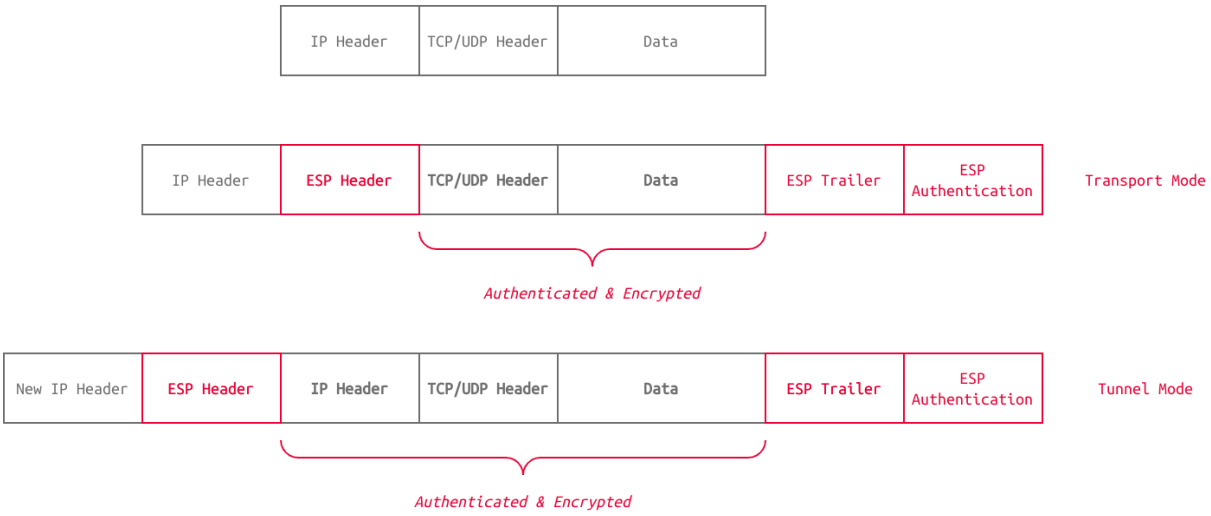


The AH protocol is suitable if providing authentication and integrity is enough without confidentiality. It is worth mentioning that AH is optional in IPsec-v3; however, it is mandatory to implement in IPsec-v2.

Encapsulating Security Payload (ESP): Encapsulating Security Payload (ESP) provides encryption in addition to authentication and integrity. It works in two modes:

1. Transport Mode: Provides security (confidentiality and integrity) for the TCP/UDP header and data.

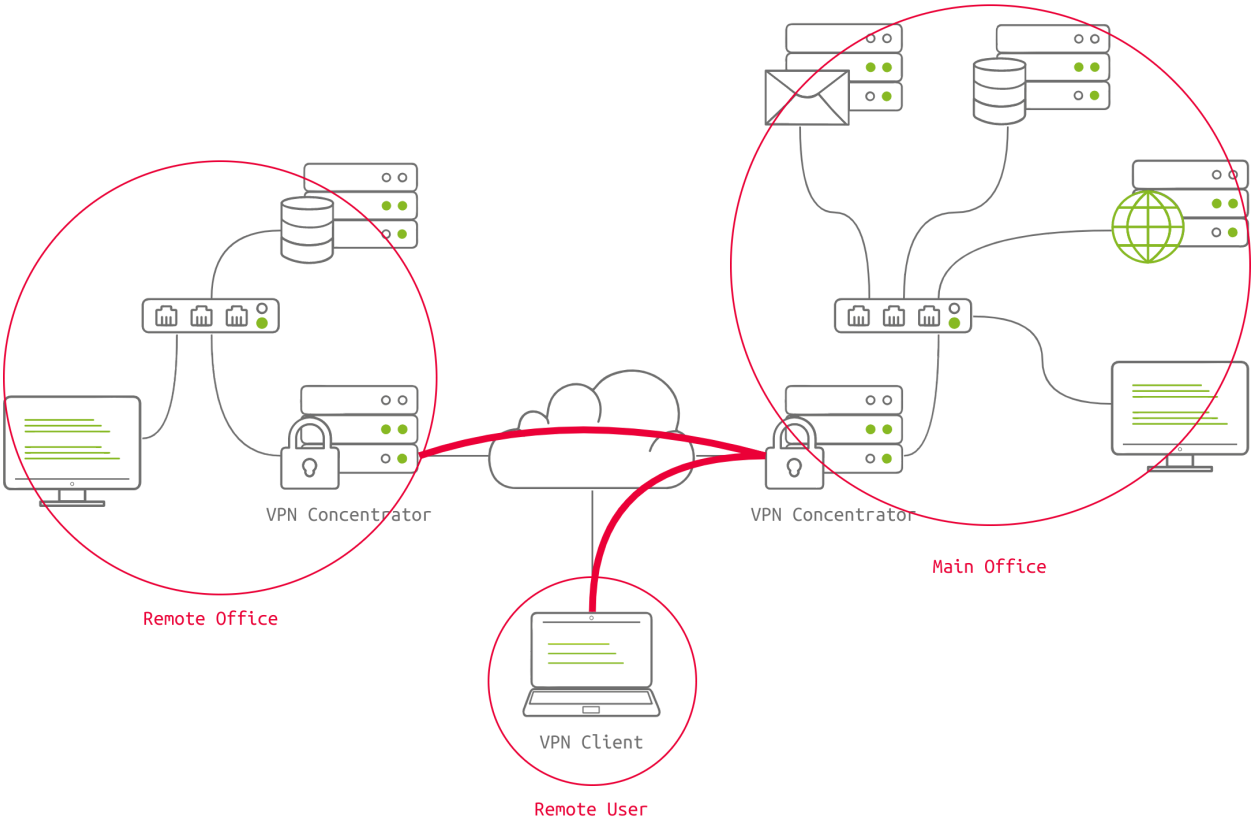
2. Tunnel Mode: Provides security (confidentiality and integrity) for the IP header,TCP/UDP header, and data.



VPN: When the TCP/IP protocol was designed, security requirements such as confidentiality and integrity were not a design target. In contrast, availability was the priority as one of the purposes of the Internet is to withstand a nuclear attack, as is evident by the routing protocols adapting quickly when a link goes down. But we need to allow a corporation to use the existing Internet infrastructure to connect its offices securely. The answer lies in setting up a VPN.

A Virtual Private Network (VPN) makes it possible to establish a private connection over a public network. In other words, we can establish a secure connection over an insecure infrastructure.

For instance, in the figure below, we can see a remote office and a remote user connected over a VPN to the main office. A VPN connection requires a VPN client and a VPN server or concentrator. All the traffic between the VPN client and server is encrypted.



The two most common protocols used to establish VPN connections are:

1. IPsec
2. SSL/TLS

IPsec's ESP is a perfect protocol for setting up secure tunnels between different networks or a computer and a network. ESP can provide security and integrity of all data transmitted between two points; moreover, even the IP address can be hidden in tunnel mode. Note that the system must be behind a VPN concentrator for the IP address to be hidden. Cisco VPN systems offer IPsec.

Although SSL was created to secure HTTP traffic, SSL/TLS has found its way to establish secure VPN connections with OpenVPN. Using various tools and libraries built around TLS, OpenVPN offers different authentication and encryption mechanisms to establish VPN connections.

Some older protocols that can be used to establish VPN connections are no longer considered secure. One example is Point to Point Tunneling Protocol (PPTP), which is no longer considered secure.

Answer the questions below

> What does ESP stand for?

Encapsulating Security Payload

> Which protocol does the Cisco VPN client use to establish a VPN connection?

IPsec

> Which protocol does the OpenVPN project use for encryption and authentication?

SSL/TLS